

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_4b294c39-6a7\agent-ac707d62e537c181c.jsonl`
- SHA-256: `729a008197d94faa1a8847daa25f364797df094880ea386aefced9f2422ad55d`
- Source modified: `2026-06-22T18:51:29+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_4b294c39-6a7`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T18:49:26.605Z)

SPA repo at C:/Users/avidu/Projects/khelsutra, web/src. Verify the USER-FACING journey in the KhelSutra Studio SPA: load a video (in-UI file upload via web/src/hooks/useUpload + IngestPanel, OR paste a gs:// URI), watch processing (useIngestJob poll), see/query the detected rallies, pick rallies, BUILD THE REEL (App.tsx onBuild ? compileReel, now async-poll #108), preview/DOWNLOAD it. Also the compute picker toggle (IngestPanel "On my computer"/"In the cloud" + cost-confirm) and the 6 supported locales (web/src/i18n). Return the step-by-step user journey + which screens/controls do what. Report only VERIFIED facts with file:line citations ? this doc must reflect SHIPPED state, never aspiration. If something is NOT implemented, say so.

2. assistant (2026-06-22T18:49:29.340Z)

I'll verify the user-facing journey in the KhelSutra Studio SPA. Let me start by pulling the repo and exploring the relevant files.

3. user (2026-06-22T18:49:33.391Z)

Updating ed33a51..bc4e7ba

Fast-forward

```
NEXT_STEPS.md | 20 ++--
README.md | 2 +-
docs/MULTILINGUAL_EXPANSION.md | 2 +-
docs/README.md | 16 ++-
docs/archives/FRIENDLY_AND_MULTILINGUAL.md | 7 +-
site/public/flyer-da.html | 171 +++++
site/public/flyer-es.html | 171 +++++
site/public/flyer-id.html | 171 +++++
site/public/img/studio-da.png | Bin 0 -> 69965 bytes
site/public/img/studio-es.png | Bin 0 -> 73131 bytes
site/public/img/studio-id.png | Bin 0 -> 68956 bytes
site/public/index.html | 2 +-
site/public/own-model.html | 2 +-
site/server.js | 3 +
site/src/assets.test.js | 33 +++--
15 files changed, 566 insertions(+), 34 deletions(-)
create mode 100644 site/public/flyer-da.html
create mode 100644 site/public/flyer-es.html
---BRANCH---
master
bc4e7ba docs: close out the Friendly & Multilingual chapter ? gates cleared, localization 100% (#110)
0e58427 feat(site): es/da/id academy flyers (parity with en/kn/hi) ? P3.8 (#109)
ed33a51 feat(build): poll the async /api/compile job for the reel (#329) (#108)
```

4. user (2026-06-22T18:49:33.638Z)

No files found

5. assistant (2026-06-22T18:49:36.213Z)

The repo is up to date. The Glob with explicit path returned nothing ? let me adjust the path handling and explore the web/src structure.

6. user (2026-06-22T18:49:37.769Z)

```
web/src/App.tsx
web/src/api/client.test.ts
web/src/api/client.ts
web/src/api/locale.test.ts
web/src/api/types.ts
web/src/api/upload.test.ts
web/src/components/CompileResult.tsx
web/src/components/IngestPanel.test.tsx
web/src/components/IngestPanel.tsx
web/src/components/LanguageSwitcher.tsx
web/src/components/QueryBar.examples.test.ts
web/src/components/QueryBar.test.tsx
web/src/components/QueryBar.tsx
web/src/components/RallyGrid.tsx
web/src/components/SelectionFooter.test.tsx
web/src/components/SelectionFooter.tsx
web/src/components/SetupWizard.test.tsx
web/src/components/SetupWizard.tsx
web/src/components/StatusChip.tsx
web/src/components/TapFilters.test.tsx
web/src/components/TapFilters.tsx
web/src/components/a4.touch.test.tsx
web/src/components/a8.status.test.tsx
web/src/components/build-error.test.tsx
web/src/components/upload.test.tsx
web/src/hooks/useIngestJob.test.ts
web/src/hooks/useIngestJob.ts
web/src/hooks/useRallySelection.ts
web/src/hooks/useUpload.test.ts
web/src/hooks/useUpload.ts
web/src/i18n/catalog-invariants.test.ts
web/src/i18n/hygiene.test.ts
web/src/i18n/i18n.test.tsx
web/src/i18n/index.ts
web/src/lib/errorText.test.ts
web/src/lib/errorText.ts
web/src/lib/errors.test.ts
web/src/lib/errors.ts
web/src/lib/floor.test.ts
web/src/lib/floor.ts
web/src/lib/format.test.ts
web/src/lib/format.ts
web/src/lib/log.ts
web/src/lib/query.test.ts
web/src/lib/query.ts
web/src/lib/selection.test.ts
web/src/lib/selection.ts
web/src/main.tsx
web/src/test/setup.ts
web/src/vite-env.d.ts
```

7. assistant (2026-06-22T18:49:40.255Z)

Now let me read the core files. Let me start with App.tsx as the main orchestrator.

8. user (2026-06-22T18:49:40.641Z)

```
1      import { useEffect, useMemo, useState } from "react";
2      import { useTranslation, Trans } from "react-i18next";
3      import { useRallySelection } from "../hooks/useRallySelection";
4      import { RallyGrid } from "../components/RallyGrid";
5      import { QueryBar } from "../components/QueryBar";
6      import { TapFilters } from "../components/TapFilters";
7      import { SelectionFooter } from "../components/SelectionFooter";
8      import { LanguageSwitcher } from "../components/LanguageSwitcher";
9      import { CompileResult, type BuildState } from "../components/CompileResult";
10     import { IngestPanel } from "../components/IngestPanel";
11     import { SetupWizard, isWizardDone } from "../components/SetupWizard";
12     import { runQuery, type ParsedQuery } from "../lib/query";
13     import { floorRisk } from "../lib/floor";
14     import { reelFilename } from "../lib/format";
15     import { classifyError } from "../lib/errors";
16     import { errorText } from "../lib/errorText";
17     import { log, newCorrelationId } from "../lib/log";
18     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
19     import type { EngineConfig, RallySegment } from "../api/types";
20
21     /** One compact, structured description of how a query was understood (for the trace
log). */
22     function parseTrace(q: ParsedQuery) {
23         return {
24             raw: q.raw,
25             recognized: q.recognized,
26             filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
27             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
28             limit: q.limit ?? null,
29             unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
30         };
31     }
32
33     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
34     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
35     const DEMO_RALLIES: RallySegment[] = [
36         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
37         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
38         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
39         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
40         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
41         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
42     ];
43
44     type LoadState =
45         | { status: "demo" }
46         | { status: "loading" }
47         | { status: "loaded"; count: number }
48         | { status: "error"; error: string };
49
50     export default function App() {
51         const { t, i18n } = useTranslation();
52         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
53         // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
```

```

processed video
54     // without a reload (it also rewrites ?video= so a refresh is stable).
55     const [videoId, setVideoId] = useState<string | null>()
56     () => new URLSearchParams(window.location.search).get("video"),
57     );
58
59     const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
60     const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
61     const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
62     const [reelName, setReelName] = useState("highlights");
63     const [build, setBuild] = useState<BuildState>({ status: "idle" });
64     // P2c: the first-run wizard shows once per browser, before the ingest panel, when
nothing's loaded.
65     const [wizardDone, setWizardDone] = useState(isWizardDone);
66
67     // Display order is separate from selection: a query can re-sort the cards while the
shared
68     // selection set (sel) stays the single source of truth for what lands in the reel.
69     const sel = useRallySelection(rallies, "all");
70
71     // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
72     useEffect(() => {
73         if (!videoId) return;
74         const cid = newCorrelationId();
75         log("info", "rallies.load_start", { cid, video_id: videoId });
76         // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
77         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
78         setLoadState({ status: "loading" });
79         let alive = true;
80         listRallies(videoId, cid)
81             .then((rs) => {
82                 if (!alive) return;
83                 setRallies(rs);
84                 setOrder(rs);
85                 sel.apply(rs.map((r) => r.id));
86                 setLoadState({ status: "loaded", count: rs.length });
87                 log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
88             })
89             .catch((err) => {
90                 if (!alive) return;
91                 // Friendly, localized ? same shared mapping as build/ingest (A5): a load
failure on a slow or
92                 // down engine reads "is it running?", not a raw "ApiError: Network error
calling POST ?".
93                 const e = classifyError(err);
94                 setLoadState({ status: "error", error: errorText(t, e) });
95                 log("error", "rallies.load_failed", { cid, video_id: videoId, code: e.code,
detail: e.detail, error: String(err) });
96             });
97         return () => {
98             alive = false;
99         };
100         // videoId is stable for the page; sel.apply is a stable callback.
101         // eslint-disable-next-line react-hooks/exhaustive-deps
102     }, [videoId]);
103
104     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
105     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
106     const [cfg, setCfg] = useState<EngineConfig | null>(null);
107     useEffect(() => {
108         let alive = true;

```

```

109     getConfig()
110     .then((c) => alive && setCfg(c))
111     .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
112     return () => {
113         alive = false;
114     };
115 }, []);
116 const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
117
118 // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
119 const ordinalOf = useMemo(() => {
120     const m = new Map<number, number>();
121     [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
122 m.set(r.id, i + 1));
123     return (id: number) => m.get(id) ?? 0;
124 }, [rallies]);
125
126 // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
127 whole call.
128 const onBuild = async () => {
129     if (build.status === "building") return;
130     const cid = newCorrelationId();
131     const filename = reelFilename(reelName);
132     const ids = [...sel.selected];
133     if (!videoId) {
134         log("info", "build.no_video", { cid, selected: ids.length });
135         setBuild({
136             status: "error",
137             error: t("build.noVideo"),
138         });
139         return;
140     }
141     setBuild({ status: "building" });
142     log("info", "build.start", {
143         cid,
144         video_id: videoId,
145         filename,
146         selected: ids.length,
147         total_sec: Math.round(sel.totalSec),
148         below_floor: warning?.atRisk.length ?? 0,
149     });
150     try {
151         // Pass the active UI locale so the engine localizes the reel's branding watermark
152         (LW1).
153         const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
154         ?? i18n.language);
155         const url = highlightUrl(videoId, filename);
156         setBuild({ status: "done", url, filename });
157         log("info", "build.done", { cid, video_id: videoId, filename, filepath:
158         res.filepath, url });
159     } catch (err) {
160         // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
161         "is the engine
162         // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
163         (A5).
164         const status = err instanceof ApiError ? err.status : undefined;
165         const e = classifyError(err);
166         setBuild({ status: "error", error: errorText(t, e) });
167         log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
168         e.code, detail: e.detail, error: String(err) });
169     }
170 };
171
172 // Every query application is ONE correlatable journey: a `query.parse` line records
173 exactly how

```

```

165      // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
166      // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
167      const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
168          const cid = newCorrelationId();
169          log("info", "query.parse", { cid, source, ...parseTrace(q) });
170
171          // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
172          // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
173          if (q.unavailable.length > 0) {
174              log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
175          }
176
177          if (!q.recognized) {
178              log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
179              return; // empty / unparseable ? leave the user's curation untouched
180          }
181
182          try {
183              const { ordered, selectedIds } = runQuery(rallies, q);
184              setOrder(ordered);
185              sel.apply(selectedIds);
186              log("info", "query.apply", {
187                  cid,
188                  source,
189                  filters: q.filters.length,
190                  sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
191                  limit: q.limit ?? null,
192                  total: rallies.length,
193                  selected: selectedIds.length,
194                  selected_ids: selectedIds,
195                  dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
196              });
197              if (selectedIds.length === 0) {
198                  // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
199                  log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
200              }
201          } catch (err) {
202              // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
203              log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
204          }
205      };
206
207      // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
208      const onToggle = (id: number) => {
209          log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
210          sel.toggle(id);
211      };
212      const onBulk = (op: "select_all" | "clear" | "invert") => {
213          const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
214          log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
215          if (op === "select_all") sel.selectAll();
216          else if (op === "clear") sel.clearAll();
217          else sel.invert();
218      };

```

```

219
220     return (
221         <main className="min-h-screen bg-paper text-ink pb-24">
222             <header className="bg-ink text-white">
223                 <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
224                 py-4">
225                     <span className="text-xl font-extrabold tracking-tight">
226                         Khel<span className="text-green">?</span>Sutra <span className="font-
227                         semibold text-lime">Studio</span>
228                     </span>
229                     <LanguageSwitcher />
230                 </div>
231             </header>
232
233             <section className="mx-auto max-w-5xl px-6 py-10">
234                 <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
235                 semibold text-green">
236                     {t("app.badge")}
237                 </span>
238                 <h1 className="mt-4 text-3xl font-extrabold tracking-
239                 tight">{t("app.title")}</h1>
240                 <p className="mt-2 max-w-2xl text-ink/70">
241                     <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
242                 </p>
243
244                 {/* P2c first-run wizard: before anything is loaded, walk a new user through AI-
245                 key / compute /
246                 videos (from /api/capabilities). Shows once per browser; then the ingest
247                 panel takes over. */}
248                 {!videoId && !wizardDone && <SetupWizard onClose={() => setWizardDone(true)} />}
249
250                 {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
251                 bucket URI / host
252                 path ? process ? load. Once a video_id exists, the existing back-half takes
253                 over. */}
254                 {!videoId && wizardDone && (
255                     <IngestPanel
256                         onLoad={id => {
257                             setVideoId(id);
258                             window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
259                         }}
260                     </>
261                 )}
262
263                 <ModeBanner load={loadState} videoId={videoId} />
264
265                 <div className="mt-6">
266                     <QueryBar onApply={onApplyQuery} />
267                     {/* A9: localized tap controls for users who can't type the English query
268                     syntax. */}
269                     <TapFilters onApply={onApplyQuery} />
270                 </div>
271
272                 <div className="mt-6 flex flex-wrap gap-2 text-sm">
273                     <button type="button" onClick={() => onBulk("select_all")} className="inline-
274                     flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
275                     green">{t("bulk.selectAll")}</button>
276                     <button type="button" onClick={() => onBulk("clear")} className="inline-flex
277                     min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
278                     green">{t("bulk.clear")}</button>
279                     <button type="button" onClick={() => onBulk("invert")} className="inline-flex
280                     min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
281                     green">{t("bulk.invert")}</button>
282                 </div>
283             </section>
284         </main>
285     )

```

```

269         <div className="mt-5">
270             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
271         </div>
272
273         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
274     </section>
275
276     <SelectionFooter
277         count={sel.count}
278         totalSec={sel.totalSec}
279         reelName={reelName}
280         onReelNameChange={setReelName}
281         onBuild={onBuild}
282         building={build.status === "building"}
283         warning={warning}
284     />
285 </main>
286 );
287 }
288
289 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
290     if (load.status === "demo") {
291         return (
292             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
293                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
294             </p>
295         );
296     }
297     if (load.status === "loading") {
298         return (
299             <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
300                 <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
301             </p>
302         );
303     }
304     if (load.status === "error") {
305         return (
306             <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
307                 <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
308             </p>
309         );
310     }
311     return (
312         <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
313             <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
314         </p>
315     );
316 }
317

```

9. assistant (2026-06-22T18:49:42.837Z)

Now let me read the upload hook, IngestPanel, useIngestJob, and the API client in parallel.

10. user (2026-06-22T18:49:43.771Z)

```

1 // Drives the in-UI file UPLOAD half of ingest (tracker T4): chunk a local file up to
the engine's
2 // /api/upload/* endpoints, report byte progress, then hand the returned server PATH to
the existing

```



```

3      // processing back-half (useIngestJob.submit). Alpha users have a FILE, not a gs:// URI
? this is the
4      // keystone that lets them point the tool at it from the browser.
5      //
6      // Kept i18n-FREE (returns stable IngestErrorCodes via classifyError, not prose) so it
unit-tests
7      // without a translation context ? IngestPanel maps codes to localized strings via
errorText. Abort
8      // is an AbortController: cancel/unmount/a superseding pick stops the in-flight upload
and the
9      // orchestrator best-effort DELETES the server-side partial.
10     import { useCallback, useEffect, useRef, useState } from "react";
11     import { uploadFile } from "../api/client";
12     import { classifyError, type IngestError } from "../lib/errors";
13     import { log, newCorrelationId } from "../lib/log";
14
15     export type UploadPhase =
16       | { status: "idle" }
17       | { status: "uploading"; pct: number; filename: string }
18       | { status: "error"; error: IngestError };
19
20     export function useUpload(onComplete: (path: string) => void) {
21       const [phase, setPhase] = useState<UploadPhase>({ status: "idle" });
22       const ctrlRef = useRef<AbortController | null>(null);
23       // Keep the latest onComplete without making start depend on it (a parent re-render
won't restart).
24       const onCompleteRef = useRef(onComplete);
25       onCompleteRef.current = onComplete;
26
27       const stop = useCallback(() => {
28         ctrlRef.current?.abort();
29         ctrlRef.current = null;
30       }, []);
31
32       // Abort any in-flight upload on unmount.
33       useEffect(() => stop, [stop]);
34
35       const start = useCallback(
36         (file: File) => {
37           // A 0-byte "video" is never valid ? reject before touching the network (reuse
invalidUri copy).
38           if (!file || file.size === 0) {
39             setPhase({ status: "error", error: { code: "invalidUri" } });
40             return;
41           }
42
43           stop(); // supersede any prior upload
44           const ctrl = new AbortController();
45           ctrlRef.current = ctrl;
46           const cid = newCorrelationId();
47           setPhase({ status: "uploading", pct: 0, filename: file.name });
48           log("info", "upload.submit_start", { cid, filename: file.name, size: file.size });
49
50           void (async () => {
51             try {
52               const res = await uploadFile(file, {
53                 cid,
54                 signal: ctrl.signal,
55                 onProgress: ({ sentBytes, totalBytes }) => {
56                   if (ctrl.signal.aborted) return;
57                   const pct = totalBytes > 0 ? Math.min(100, Math.round((sentBytes /
totalBytes) * 100)) : 0;
58                   setPhase({ status: "uploading", pct, filename: file.name });
59                 },
60               });

```

```

61         if (ctrl.signal.aborted) return;
62         setPhase({ status: "idle" }); // hand off ? the engine processing phase takes
over
63         onCompleteRef.current(res.path);
64     } catch (err) {
65         // User-cancelled (abort) is silent, not an error.
66         if (ctrl.signal.aborted || (err instanceof DOMException && err.name ===
"AbortError")) return;
67         setPhase({ status: "error", error: classifyError(err) });
68         log("error", "upload.failed", { cid, filename: file.name, error: String(err)
});
69     }
70     })();
71     },
72     [stop],
73 );
74
75     const cancel = useCallback(() => {
76         stop();
77         setPhase({ status: "idle" });
78         log("info", "upload.cancel", {});
79     }, [stop]);
80
81     const reset = useCallback(() => {
82         stop();
83         setPhase({ status: "idle" });
84     }, [stop]);
85
86     const busy = phase.status === "uploading";
87     return { phase, start, cancel, reset, busy };
88 }
89

```

11. user (2026-06-22T18:49:44.727Z)

```

1     // The front-half ingest screen (appliance S1/S2; tracker B-P2 + T4). Two ways to point
the tool at a
2     // video: UPLOAD a local file (primary ? alpha users have a file, not a URI; chunked to
the engine's
3     // /api/upload/* endpoints) or paste a `gs://` bucket URI / host path (secondary). Both
converge on
4     // the SAME back-half: POST /api/process ? poll GET /api/jobs/{id} ? hand the video_id
to App. Honest
5     // by construction: the engine reports coarse phase WORDS (not a %), and a success-
shaped failure
6     // (done but no rallies) is shown, never silently treated as a load.
7     import { useEffect, useState } from "react";
8     import type { ChangeEvent, FormEvent } from "react";
9     import { useTranslation } from "react-i18next";
10    import { getCapabilities } from "../api/client";
11    import { useIngestJob } from "../hooks/useIngestJob";
12    import { useUpload } from "../hooks/useUpload";
13    import { errorText } from "../lib/errorText";
14    import { log } from "../lib/log";
15    import { StatusChip, Spinner } from "../StatusChip";
16
17    type Compute = "local" | "cloud";
18
19    interface Props {
20        /** Called with the engine's video_id once a submitted job finishes successfully. */
21        onLoaded: (videoId: string) => void;
22    }
23
24    export function IngestPanel({ onLoaded }: Props) {
25        const { t, i18n } = useTranslation();

```

```

26     const [uri, setUri] = useState("");
27     const { phase, submit, cancel: cancelIngest, reset: resetIngest, busy: ingestBusy } =
useIngestJob(onLoaded);
28     // The UI locale recorded with the submission (PMF analytics + the localized reel
watermark).
29     const locale = i18n.resolvedLanguage ?? i18n.language;
30
31     // Compute-picker (PACKAGING compute-picker, item 3): if this server can dispatch to a
cloud GPU
32     // (capabilities.deployment.cloud_available), let the user choose where DETECT runs.
Default LOCAL
33     // (free, no surprise cost); CLOUD requires an explicit cost acknowledgement before it
can run.
34     const [cloudAvailable, setCloudAvailable] = useState(false);
35     const [compute, setCompute] = useState<Compute>("local");
36     const [cloudAck, setCloudAck] = useState(false);
37     useEffect(() => {
38         let alive = true;
39         getCapabilities()
40             .then((c) => alive && setCloudAvailable(c.deployment?.cloud_available === true))
41             .catch((err) => log("warn", "capabilities.unavailable", { error: String(err) }));
42         return () => {
43             alive = false;
44         };
45     }, []);
46     // Only send a choice when the picker is actually shown; otherwise omit it (? server
default).
47     const sendCompute = cloudAvailable ? compute : undefined;
48     // CLOUD selected but the cost note not yet accepted ? block submit (don't silently
run local, and
49     // don't dispatch a paid job without consent).
50     const cloudBlocked = cloudAvailable && compute === "cloud" && !cloudAck;
51
52     // On upload completion, feed the returned server PATH straight into the engine
processing back-half.
53     // The arrow is recreated each render (useUpload reads onCompleteRef.current), so
sendCompute is fresh.
54     const upload = useUpload((path) => submit(path, locale, sendCompute));
55     const busy = ingestBusy || upload.busy;
56     // Disable the submit surfaces (file picker + URI form) while busy OR awaiting the
cloud cost ack.
57     const inputsDisabled = busy || cloudBlocked;
58
59     const onSubmitUri = (e: FormEvent) => {
60         e.preventDefault();
61         if (cloudBlocked) return;
62         submit(uri, locale, sendCompute);
63     };
64     const onFile = (e: ChangeEvent<HTMLInputElement>) => {
65         const file = e.target.files?.[0];
66         // Symmetric with onSubmitUri: never start a cloud upload before the cost is acked
(the disabled
67         // input already blocks this; this is belt-and-suspenders).
68         if (file && !cloudBlocked) upload.start(file);
69         e.target.value = ""; // let re-picking the same file fire change again
70     };
71     const cancelAll = () => {
72         upload.cancel();
73         cancelIngest();
74     };
75     const resetAll = () => {
76         upload.reset();
77         resetIngest();
78     };
79

```

```

80      // Shared mapping with the build flow (lib/errorText): an `engine` failure carries the
engine's own
81      // message (a 400 scheme/path/extension detail, a worker error); every other code maps
to a friendly
82      // localized line. Surface whichever half failed (upload or engine) ? they're mutually
exclusive.
83      const errorState = upload.phase.status === "error" ? upload.phase.error : phase.status
=== "error" ? phase.error : null;
84      const errorMsg = errorState ? errorText(t, errorState) : null;
85
86      return (
87        <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
88          <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
89          <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
90
91          {/* COMPUTE PICKER (item 3): shown only when this server can dispatch to a cloud
GPU. Default
92             LOCAL; CLOUD reveals a cost note + a required acknowledgement before any
submit can run. */}
93          {cloudAvailable && (
94            <fieldset className="mt-4 rounded-xl border border-black/10 p-3" data-
testid="compute-picker">
95              <legend className="px-1 text-xs font-semibold uppercase tracking-wide text-
ink/40">
96                {t("ingest.compute.legend")}
97              </legend>
98              <div className="mt-1 flex flex-wrap gap-2">
99                {[["local", "cloud"] as const].map((opt) => (
100                  <button
101                    key={opt}
102                    type="button"
103                    aria-pressed={compute === opt}
104                    disabled={busy}
105                    onClick={() => setCompute(opt)}
106                    className={
107                      "min-h-11 rounded-xl border px-4 py-2 text-sm font-semibold
disabled:opacity-40 " +
108                      (compute === opt
109                        ? "border-green bg-green/10 text-ink"
110                        : "border-black/15 text-ink/70 hover:border-green")}
111                    }
112                  >
113                    {t(`ingest.compute.${opt}`)}
114                  </button>
115                )]}
116              </div>
117              {compute === "cloud" && (
118                <div className="mt-3" data-testid="compute-cost">
119                  <p className="text-xs text-ink/60">{t("ingest.compute.cloudCost")}</p>
120                  <label className="mt-2 flex items-center gap-2 text-sm text-ink/80">
121                    <input
122                      type="checkbox"
123                      checked={cloudAck}
124                      disabled={busy}
125                      onChange={(e) => setCloudAck(e.target.checked)}
126                      className="h-4 w-4"
127                    />
128                    {t("ingest.compute.ack")}
129                  </label>
130                  <cloudBlocked && (
131                    <p className="mt-1 text-xs text-amber-700" data-testid="compute-ack-
required">
132                      {t("ingest.compute.ackRequired")}
133                    </p>

```

```

134         )}
135     </div>
136     )}
137 </fieldset>
138 )}
139
140 {/* PRIMARY: upload a local file (T4). A native file input, visually a button
(244px, A4). */}
141 <div className="mt-4">
142     <label
143         className={
144             // focus-within makes the visible button show a focus ring when the (sr-
only) input is
145             // focused ? a keyboard user needs a visible focus indicator (WCAG 2.4.7).
146             "inline-flex min-h-11 items-center rounded-xl bg-ink px-4 py-2 text-sm font-
semibold text-white hover:bg-ink2 focus-within:outline-none focus-within:ring-2 focus-
within:ring-green focus-within:ring-offset-2 " +
147             (inputsDisabled ? "pointer-events-none opacity-40" : "cursor-pointer")
148         }
149     >
150         {t("ingest.chooseFile")}
151         <input
152             type="file"
153             accept=".mp4,.mov,video/mp4,video/quicktime"
154             aria-label={t("ingest.uploadAria")}
155             disabled={inputsDisabled}
156             onChange={onFile}
157             className="sr-only"
158         />
159     </label>
160     <p className="mt-2 text-xs text-ink/40">{t("ingest.fileHint")}</p>
161 </div>
162
163 {/* SECONDARY: or point at a cloud bucket / host path. */}
164 <div className="mt-4 border-t border-black/5 pt-4">
165     <p className="text-xs font-semibold uppercase tracking-wide text-
ink/40">{t("ingest.orLink")}</p>
166     <form onSubmit={onSubmitUri} className="mt-2 flex flex-wrap items-center gap-2">
167         <input
168             type="text"
169             value={uri}
170             onChange={(e) => setUri(e.target.value)}
171             placeholder={t("ingest.uriPlaceholder")}
172             aria-label={t("ingest.uriAriaLabel")}
173             disabled={inputsDisabled}
174             className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15
bg-white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
175         />
176         <button
177             type="submit"
178             disabled={inputsDisabled}
179             className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
180         >
181             {phase.status === "submitting" ? t("ingest.submitting") :
t("ingest.submit")}
182         </button>
183     </form>
184     <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
185 </div>
186
187 {/* Live status (aria-live so screen readers hear progress) as icon+text chips ?
never icon-only
188     (A8). Upload shows a determinate % bar; the engine phase is indeterminate
(phase words). */}

```

```

189     <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
190         {upload.phase.status === "uploading" && (
191             <div data-testid="upload-progress">
192                 /* Static chip label (not the per-tick %) so the aria-live region announces
once; the live
193                     % rides the progressbar's aria-valuenow + a sighted-only label (A8/T4).
*/
194                 <StatusChip tone="info" icon={<Spinner />}
label={t("ingest.uploadingStatus")} />
195                 <div className="mt-2 flex max-w-sm items-center gap-2">
196                     <div
197                         className="h-1.5 flex-1 overflow-hidden rounded-full bg-black/10"
198                         role="progressbar"
199                         aria-label={t("ingest.uploadProgress")}
200                         aria-valuenow={upload.phase.pct}
201                         aria-valuemin={0}
202                         aria-valuemax={100}
203                     >
204                         <div className="h-full bg-green transition-all" style={{ width:
`$${upload.phase.pct}%` }} />
205                     </div>
206                     <span aria-hidden="true" className="text-xs tabular-nums text-ink/50">
207                         {upload.phase.pct}%
208                     </span>
209                 </div>
210                 <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
211                 <button
212                     type="button"
213                     onClick={cancelAll}
214                     className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
215                 >
216                     {t("ingest.cancel")}
217                 </button>
218             </div>
219         )}
220     {ingestBusy && (
221         <div>
222             <StatusChip
223                 tone="info"
224                 icon={<Spinner />}
225                 label={
226                     t("ingest.working") +
227                     (phase.status === "polling" && phase.progress
228                         ? ` · ${t("ingest.progress", { progress: phase.progress })}`
229                         : "")
230                 }
231             />
232             /* Reassurance for slow uplinks: the job runs on the server, so it's safe
to wait. */
233             <p className="mt-2 text-xs text-ink/50">{t("ingest.reassure")}</p>
234             <button
235                 type="button"
236                 onClick={cancelAll}
237                 className="mt-2 inline-flex min-h-11 items-center rounded-xl border
border-black/15 px-4 text-sm hover:border-green"
238             >
239                 {t("ingest.cancel")}
240             </button>
241         </div>
242     )}
243     {errorMsg && (
244         <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
245             <StatusChip tone="error" icon="?" label={t("ingest.statusFailed")} />

```

```

246         <p className="mt-2">{errorMsg}</p>
247         <button
248             type="button"
249             onClick={resetAll}
250             className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
251         >
252             {t("ingest.retry")}
253         </button>
254     </div>
255     )}
256 </div>
257 </section>
258 );
259 }
260

```

12. user (2026-06-22T18:49:44.834Z)

```

1      // Drives the front-half ingest: submit a video to POST /api/process, then poll GET
/api/jobs/{id}
2      // until it finishes, and hand the resulting video_id to the existing rally-loading
back-half.
3      //
4      // Kept i18n-FREE (returns stable IngestErrorCodes, not prose) so it unit-tests without
a
5      // translation context ? IngestPanel maps codes to localized strings. The poll loop is a
RECURSIVE
6      // setTimeout (never setInterval): each tick waits for the prior getJob to resolve, so a
slow engine
7      // can't pile up overlapping requests. It backs off, gives up after a ceiling, and is
fully
8      // cancellable on unmount / user-cancel / a superseding submit.
9      import { useCallback, useEffect, useRef, useState } from "react";
10     import { getJob, processVideo } from "../api/client";
11     import { classifyError, type IngestError } from "../lib/errors";
12     import { log, newCorrelationId } from "../lib/log";
13     import type { JobStatus } from "../api/types";
14
15     const POLL_MIN_MS = 1000;
16     const POLL_MAX_MS = 5000;
17     const POLL_BACKOFF = 1.5;
18     const POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' job won't poll forever
19
20     export type IngestPhase =
21     | { status: "idle" }
22     | { status: "submitting" }
23     | { status: "polling"; progress: string | null }
24     | { status: "done"; videoId: string }
25     | { status: "error"; error: IngestError };
26
27     interface Run {
28         cancelled: boolean;
29         timer?: ReturnType<typeof setTimeout>;
30     }
31
32     export function useIngestJob(onLoaded?: (videoId: string) => void) {
33         const [phase, setPhase] = useState<IngestPhase>({ status: "idle" });
34         const runRef = useRef<Run | null>(null);
35         // Keep the latest onLoaded without making submit depend on it (a parent re-render
won't restart polls).
36         const onLoadedRef = useRef(onLoaded);
37         onLoadedRef.current = onLoaded;
38
39         const stop = useCallback(() => {

```

```

40     const run = runRef.current;
41     if (run) {
42         run.cancelled = true;
43         if (run.timer) clearTimeout(run.timer);
44     }
45     runRef.current = null;
46 }, []);
47
48 // Cancel any in-flight poll loop when the component unmounts (App.tsx:80-82 alive-
flag idiom).
49 useEffect(() => stop, [stop]);
50
51 const submit = useCallback(
52     (rawUri: string, locale?: string, compute?: string) => {
53         const uri = rawUri.trim();
54         if (!uri) {
55             setPhase({ status: "error", error: { code: "invalidUri" } });
56             return;
57         }
58
59         stop(); // supersede any prior run
60         const run: Run = { cancelled: false };
61         runRef.current = run;
62
63         const cid = newCorrelationId();
64         const startedAt = Date.now();
65         setPhase({ status: "submitting" });
66         log("info", "ingest.submit_start", { cid, uri });
67
68         const fail = (error: IngestError, event: string) => {
69             if (run.cancelled) return;
70             setPhase({ status: "error", error });
71             const level = error.code === "timeout" || error.code === "emptyResult" ? "warn"
: "error";
72             log(level, event, { cid, uri, code: error.code, detail: error.detail });
73         };
74
75         const poll = (jobId: string, delay: number) => {
76             run.timer = setTimeout(() => {
77                 void (async () => {
78                     if (run.cancelled) return;
79                     let job: JobStatus;
80                     try {
81                         job = await getJob(jobId, cid);
82                     } catch (err) {
83                         fail(classifyError(err), "ingest.poll_error");
84                         return;
85                     }
86                     if (run.cancelled) return;
87
88                     if (job.status === "done") {
89                         const videoId = job.video_id ?? job.result?.video_id ?? null;
90                         // Two status layers: a `done` job with a `failed` pipeline verdict (or no
video_id) is
91                         // a success-shaped FAILURE ? never load it, never hang.
92                         if (job.result?.status === "failed" || !videoId) {
93                             fail({ code: "emptyResult" }, "ingest.empty_result");
94                             return;
95                         }
96                         setPhase({ status: "done", videoId });
97                         log("info", "ingest.done", { cid, uri, video_id: videoId });
98                         onLoadedRef.current?.(videoId);
99                         return;
100                     }
101                     if (job.status === "failed") {

```



```

102         fail({ code: "engine", detail: job.error ?? "job failed" },
"ingest.job_failed");
103         return;
104     }
105     // queued | running ? reflect coarse progress (free-text phase) and keep
polling.
106     setPhase({ status: "polling", progress: job.progress ?? null });
107     if (Date.now() - startedAt > POLL_CEILING_MS) {
108         fail({ code: "timeout" }, "ingest.poll_timeout");
109         return;
110     }
111     poll(jobId, Math.min(delay * POLL_BACKOFF, POLL_MAX_MS));
112     })();
113     }, delay);
114 };
115
116 void (async () => {
117     let jobId: string;
118     try {
119         const resp = await processVideo(uri, cid, locale, compute);
120         jobId = resp.job_id;
121     } catch (err) {
122         fail(classifyError(err), "ingest.submit_error");
123         return;
124     }
125     if (run.cancelled) return;
126     log("info", "ingest.queued", { cid, uri, job_id: jobId });
127     setPhase({ status: "polling", progress: null });
128     poll(jobId, POLL_MIN_MS);
129     })();
130 },
131 [stop],
132 );
133
134 const cancel = useCallback(() => {
135     stop();
136     setPhase({ status: "idle" });
137     log("info", "ingest.cancel", {});
138 }, [stop]);
139
140 const reset = useCallback(() => {
141     stop();
142     setPhase({ status: "idle" });
143 }, [stop]);
144
145 const busy = phase.status === "submitting" || phase.status === "polling";
146 return { phase, submit, cancel, reset, busy };
147 }
148

```

13. assistant (2026-06-22T18:49:46.921Z)

Now let me read the API client, types, CompileResult, and the i18n index in parallel.

14. user (2026-06-22T18:49:47.867Z)

```

1     // Typed client for the engine FastAPI. Default = SAME-ORIGIN relative `/api/*` (the
single-origin
2     // appliance reverse-proxies UI + engine behind one origin; in dev Vite proxies `/api` ?
:8000).
3     // Every call carries a correlation id (header + logs) so a journey is traceable end to
end.
4     import { log, newCorrelationId } from "../lib/log";
5     import type {
6         RallySegment,

```

```

7      QueryResponse,
8      CompileResult,
9      EngineConfig,
10     EngineCapabilities,
11     ProcessResponse,
12     JobStatus,
13     UploadInitResponse,
14     UploadPartResponse,
15     UploadCompleteResponse,
16 } from "./types";
17
18 /** Resolve the engine API base. Default same-origin `"/api"`. For a SPLIT deploy ? SPA
on
19 * Cloudflare Pages + the engine on a separate `api.*` cloudflared tunnel
(HOSTING_PLAN.md $2) ?
20 * set `VITE_API_BASE` at build time to the engine's API base **including the `/api`
path**
21 * (e.g. `https://api.khelsutra.guru/api`), and lock the engine's
`RALLY_CORS_ALLOW_ORIGINS` to the
22 * Pages origin + put CF Access in front of both hostnames. A trailing slash is trimmed
so
23 * `` `${base}/process` `` never double-slashes. */
24 export function apiBase(): string {
25     return (import.meta.env.VITE_API_BASE ?? "/api").replace(/\/+$/, "");
26 }
27
28 const BASE = apiBase();
29
30 export class ApiError extends Error {
31     readonly cid?: string;
32     readonly status?: number;
33     readonly body?: string;
34     constructor(message: string, opts: { cid?: string; status?: number; body?: string;
cause?: unknown } = {}) {
35         super(message);
36         this.name = "ApiError";
37         this.cid = opts.cid;
38         this.status = opts.status;
39         this.body = opts.body;
40         if (opts.cause !== undefined) (this as { cause?: unknown }).cause = opts.cause;
41     }
42 }
43
44 interface RequestOpts {
45     method?: string;
46     body?: string;
47     headers?: Record<string, string>;
48     /** Correlation id to thread through this call; auto-generated if absent. */
49     cid?: string;
50 }
51
52 async function request<T>(path: string, opts: RequestOpts = {}): Promise<T> {
53     const cid = opts.cid ?? newCorrelationId();
54     const url = `${BASE}${path}`;
55     const method = opts.method ?? "GET";
56     log("info", "api.request", { cid, method, url });
57
58     let res: Response;
59     try {
60         res = await fetch(url, {
61             method,
62             body: opts.body,
63             headers: { "content-type": "application/json", "x-correlation-id": cid,
...(opts.headers ?? {}) },
64         });

```

```

65     } catch (err) {
66         // Loud failure, never silent (LOCAL_APPLIANCE_ARCHITECTURE.md §5).
67         log("error", "api.network_error", { cid, method, url, error: String(err) });
68         throw new ApiError(`Network error calling ${method} ${url}`, { cid, cause: err });
69     }
70
71     if (!res.ok) {
72         const body = await res.text().catch(() => "");
73         log("error", "api.http_error", { cid, method, url, status: res.status, body:
body.slice(0, 500) });
74         throw new ApiError(`${method} ${url} failed: ${res.status}`, { cid, status:
res.status, body });
75     }
76
77     const data = (await res.json()) as T;
78     log("info", "api.response", { cid, method, url, status: res.status });
79     return data;
80 }
81
82 /** Submit a video (a `gs://`/`gcs://`/`s3://` bucket URI, or a path on the engine host)
for
83 * processing. POST /api/process {video_path} ? 202 {job_id, ...} [main.py:260]. Fast-
fails 400
84 * (unsupported scheme / out-of-allowed-root) or 404 (missing LOCAL file) ? surfaced as
ApiError. */
85 export async function processVideo(
86     videoPath: string,
87     cid?: string,
88     locale?: string,
89     compute?: string,
90 ): Promise<ProcessResponse> {
91     // `locale` (the UI language) is recorded for PMF analytics (which markets use the
tool). `compute`
92     // is the compute-picker choice ("local" | "cloud") ? where DETECT runs; omitted ? the
server
93     // default. Both optional + omitted when absent (JSON.stringify drops undefined).
94     return request<ProcessResponse>("/process", {
95         method: "POST",
96         body: JSON.stringify({ video_path: videoPath, locale, compute }),
97         cid,
98     });
99 }
100
101 /** Poll a submitted job. GET /api/jobs/{job_id} [main.py:303]. 404 ? unknown job_id. */
102 export async function getJob(jobId: string, cid?: string): Promise<JobStatus> {
103     return request<JobStatus>(`/jobs/${encodeURIComponent(jobId)}`, { method: "GET", cid
});
104 }
105
106 /** List a video's detected rallies. POST /api/query {video_id} ? {play_segments}
[main.py:331]. */
107 export async function listRallies(videoId: string, cid?: string):
Promise<RallySegment[]> {
108     const data = await request<QueryResponse>("/query", {
109         method: "POST",
110         body: JSON.stringify({ video_id: videoId }),
111         cid,
112     });
113     return data.play_segments ?? [];
114 }
115
116 // Compile-job poll cadence (mirrors useIngestJob): a long ffmpeg stitch must not block
the request.
117 const COMPILE_POLL_MIN_MS = 1000;
118 const COMPILE_POLL_MAX_MS = 5000;

```

```

119     const COMPILE_POLL_BACKOFF = 1.5;
120     const COMPILE_POLL_CEILING_MS = 10 * 60 * 1000; // a wedged 'running' compile won't poll
forever
121
122     /** Stitch selected rallies into one reel. ASYNC (#329): POST /api/compile returns 202 +
a job_id;
123     * the ffmpeg stitch runs on the worker and we POLL GET /api/jobs/{id} for the reel ? a
long stitch
124     * would otherwise exceed the ~100 s edge/proxy timeout (524) while the reel actually
built. Resolves
125     * with the job's CompileResult on success; throws ApiError on a failed/timed-out
compile.
126     * `locale` (the UI language) localizes the reel watermark engine-side (omitted when
absent). */
127     export async function compileReel(
128         videoId: string,
129         segmentIds: number[],
130         outputFilename: string,
131         cid?: string,
132         locale?: string,
133     ): Promise<CompileResult> {
134         const c = cid ?? newCorrelationId();
135         const submitted = await request<ProcessResponse>("/compile", {
136             method: "POST",
137             body: JSON.stringify({ video_id: videoId, segment_ids: segmentIds, output_filename:
outputFilename, locale }),
138             cid: c,
139         });
140         return pollCompileJob(submitted.job_id, c);
141     }
142
143     /** Poll a submitted compile job to its terminal state. Returns the reel result on
success; throws an
144     * ApiError for an engine job-failure, a success-shaped FAILURE (done but the stitch
failed / no URL),
145     * or a poll timeout ? so the caller's existing classifyError/errorText mapping
surfaces it like ingest. */
146     async function pollCompileJob(jobId: string, cid: string): Promise<CompileResult> {
147         const startedAt = Date.now();
148         let delay = COMPILE_POLL_MIN_MS;
149         for (;;) {
150             const job = await getJob(jobId, cid);
151             if (job.status === "done") {
152                 const result = (job.result ?? {}) as CompileResult;
153                 if (result.status === "failed" || !result.download_url) {
154                     // A real stitch failure carries an engine message ? surface it as an `engine`
detail (status +
155                     // body set) so classifyError shows it verbatim, NOT the no-status "engine
unreachable" fallback.
156                     const msg = result.message || "Highlight stitching failed.";
157                     throw new ApiError(msg, { cid, status: 502, body: msg });
158                 }
159                 return result;
160             }
161             if (job.status === "failed") {
162                 const msg = job.error || "Compile job failed.";
163                 throw new ApiError(msg, { cid, status: 502, body: msg });
164             }
165             if (Date.now() - startedAt > COMPILE_POLL_CEILING_MS) {
166                 throw new ApiError("Compile timed out.", { cid, status: 504, body: "Compile timed
out." });
167             }
168             await new Promise((resolve) => setTimeout(resolve, delay));
169             delay = Math.min(delay * COMPILE_POLL_BACKOFF, COMPILE_POLL_MAX_MS);
170         }

```

```

171 }
172
173 /** The stream/download URL for a compiled reel. GET
/api/highlights/{video_id}/{filename} [main.py:307]. */
174 export function highlightUrl(videoId: string, filename: string): string {
175     return
`${BASE}/highlights/${encodeURIComponent(videoId)}/${encodeURIComponent(filename)}`;
176 }
177
178 /** Read engine config (for the stitching floor). GET /api/config [main.py:121]. */
179 export async function getConfig(cid?: string): Promise<EngineConfig> {
180     return request<EngineConfig>("/config", { method: "GET", cid });
181 }
182
183 /** What THIS box can do ? drives the first-run wizard. GET /api/capabilities
(PACKAGING_PLAN P2c). */
184 export async function getCapabilities(cid?: string): Promise<EngineCapabilities> {
185     return request<EngineCapabilities>("/capabilities", { method: "GET", cid });
186 }
187
188 /** The shot-count floor /api/compile silently enforces ? the UI must warn, not hide
[main.py:362]. */
189 export function minShotCount(cfg: EngineConfig): number | undefined {
190     return cfg.stitching?.min_shot_count;
191 }
192
193 // ?? chunked upload (backend/api/uploads.py) ??????????????????????????????????
194 // A browser uploads a LOCAL file in sequential parts ? max_part_mb (free-tier edge
proxies cap
195 // bodies ~100 MB), then `complete` returns a server-host `path` to feed straight into
/api/process.
196
197 /** Start a chunked upload. 400 = bad extension; 413 = over the size limit
[uploads.py:64]. */
198 export async function uploadInit(filename: string, totalSizeBytes: number, cid?:
string): Promise<UploadInitResponse> {
199     return request<UploadInitResponse>("/upload/init", {
200         method: "POST",
201         body: JSON.stringify({ filename, total_size_bytes: totalSizeBytes }),
202         cid,
203     });
204 }
205
206 /** Append ONE part (raw bytes ? not JSON). Parts are strictly sequential 0,1,2,?
[uploads.py:89]. */
207 export async function uploadPart(
208     uploadId: string,
209     index: number,
210     chunk: Blob,
211     cid?: string,
212     signal?: AbortSignal,
213 ): Promise<UploadPartResponse> {
214     const c = cid ?? newCorrelationId();
215     const url = `${BASE}/upload/${encodeURIComponent(uploadId)}/part/${index}`;
216     log("info", "api.request", { cid: c, method: "PUT", url });
217     let res: Response;
218     try {
219         res = await fetch(url, {
220             method: "PUT",
221             body: chunk,
222             headers: { "content-type": "application/octet-stream", "x-correlation-id": c },
223             signal,
224         });
225     } catch (err) {
226         log("error", "api.network_error", { cid: c, method: "PUT", url, error: String(err)

```

```

});
227     throw new ApiError(`Network error calling PUT ${url}`, { cid: c, cause: err });
228 }
229 if (!res.ok) {
230     const body = await res.text().catch(() => "");
231     log("error", "api.http_error", { cid: c, method: "PUT", url, status: res.status,
body: body.slice(0, 500) });
232     throw new ApiError(`PUT ${url} failed: ${res.status}`, { cid: c, status: res.status,
body });
233 }
234 log("info", "api.response", { cid: c, method: "PUT", url, status: res.status });
235 return (await res.json()) as UploadPartResponse;
236 }
237
238 /** Finalize: returns {path, video_id} ? feed `path` to processVideo() [uploads.py:123].
*/
239 export async function uploadComplete(uploadId: string, totalParts: number, cid?:
string): Promise<UploadCompleteResponse> {
240     return
request<UploadCompleteResponse>(`/upload/${encodeURIComponent(uploadId)}/complete`, {
241         method: "POST",
242         body: JSON.stringify({ total_parts: totalParts }),
243         cid,
244     });
245 }
246
247 /** Abort an in-flight upload (best-effort cleanup of the server-side partial)
[uploads.py:147]. */
248 export async function uploadAbort(uploadId: string, cid?: string): Promise<void> {
249     await request<unknown>(`/upload/${encodeURIComponent(uploadId)}`, { method: "DELETE",
cid });
250 }
251
252 export interface UploadProgress {
253     sentBytes: number;
254     totalBytes: number;
255 }
256
257 /** Upload a local File end to end: init ? sequential parts (byte progress, abortable) ?
complete.
258 * On ANY failure or abort it best-effort DELETES the server-side partial, then
rethrows so the
259 * caller can classify it (the engine's 400/413 detail survives). Returns {path,
video_id}.
260 * Note: a per-part 413 (over the per-part limit) aborts the upload server-side ?
recovery is a
261 * fresh init (re-pick the file), which the UI's retry does. */
262 export async function uploadFile(
263     file: File,
264     opts: { cid?: string; signal?: AbortSignal; onProgress?: (p: UploadProgress) => void }
= {},
265 ): Promise<UploadCompleteResponse> {
266     const cid = opts.cid ?? newCorrelationId();
267     const init = await uploadInit(file.name, file.size, cid);
268     // Honor the engine's max_part_mb, but never let a missing/NaN value collapse the math
to NaN (which
269     // would silently upload zero parts) ? fall back to the engine's 95 MB default.
270     const partMb = Number.isFinite(init.max_part_mb) && init.max_part_mb > 0 ?
init.max_part_mb : 95;
271     const partBytes = partMb * 1024 * 1024;
272     const totalParts = Math.max(1, Math.ceil(file.size / partBytes));
273     log("info", "upload.start", { cid, filename: file.name, size: file.size, parts:
totalParts, part_mb: init.max_part_mb });
274     let sent = 0;
275     try {

```

```

276     for (let i = 0; i < totalParts; i++) {
277         if (opts.signal?.aborted) throw new DOMException("Upload aborted", "AbortError");
278         const chunk = file.slice(i * partBytes, Math.min((i + 1) * partBytes, file.size));
279         await uploadPart(init.upload_id, i, chunk, cid, opts.signal);
280         sent += chunk.size;
281         opts.onProgress?.({ sentBytes: sent, totalBytes: file.size });
282     }
283     const done = await uploadComplete(init.upload_id, totalParts, cid);
284     log("info", "upload.done", { cid, video_id: done.video_id, path: done.path, size:
done.size_bytes });
285     return done;
286 } catch (err) {
287     log("warn", "upload.aborting_partial", { cid, upload_id: init.upload_id, error:
String(err) });
288     await uploadAbort(init.upload_id, cid).catch(() => {});
289     throw err;
290 }
291 }
292

```

15. user (2026-06-22T18:49:48.825Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6      import { useTranslation } from "react-i18next";
7      import { StatusChip, Spinner } from "../StatusChip";
8
9      export type BuildState =
10         | { status: "idle" }
11         | { status: "building" }
12         | { status: "done"; url: string; filename: string }
13         | { status: "error"; error: string };
14
15      interface Props {
16         build: BuildState;
17         onDismiss: () => void;
18     }
19
20     export function CompileResult({ build, onDismiss }: Props) {
21         const { t } = useTranslation();
22         if (build.status === "idle") return null;
23
24         // Build state (A8): a visible "stitching?" card with an icon+text chip + reassurance,
so the user
25         // isn't staring at a frozen footer wondering whether anything is happening.
26         if (build.status === "building") {
27             return (
28                 <section data-testid="compile-result" className="mt-6 rounded-xl border border-
black/10 bg-paper p-4" aria-live="polite">
29                     <StatusChip tone="info" icon={<Spinner />} label={t("result.building")} />
30                     <p className="mt-2 text-sm text-ink/60">{t("result.buildingReassure")}</p>
31                 </section>
32             );
33         }
34
35         const isError = build.status === "error";
36         return (
37             <section

```

```

38         data-testid="compile-result"
39         aria-live="polite"
40         className={"mt-6 rounded-xl border p-4 " + (isError ? "border-red-200 bg-red-50" :
"border-green/30 bg-green/5")}
41     >
42         <div className="flex items-start justify-between gap-3">
43             {/* Status as icon+text (never icon-only): ? ready / ? failed (A8). */}
44             <h2 className="flex items-center gap-2 font-semibold text-ink">
45                 <span aria-hidden="true" className={isError ? "text-red-700" : "text-
green"}>{isError ? "?" : "?"}</span>
46                 {build.status === "done" ? t("result.ready") : t("result.failed")}
47             </h2>
48             <button
49                 type="button"
50                 onClick={onDismiss}
51                 aria-label={t("result.dismiss")}
52                 className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full
text-ink/50 hover:text-ink"
53             >
54                 ?
55             </button>
56         </div>
57
58         {build.status === "done" ? (
59             <div className="mt-3">
60                 {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
61                 <video
62                     data-testid="reel-video"
63                     src={build.url}
64                     controls
65                     playsInline
66                     className="w-full max-w-xl rounded-lg bg-black"
67                 />
68                 <div className="mt-3 flex flex-wrap items-center gap-2">
69                     <a
70                         href={build.url}
71                         download={build.filename}
72                         className="inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-[#04140D]"
73                     >
74                         {t("result.download", { filename: build.filename })}
75                     </a>
76                     {/* A2: localized WhatsApp share. Opens the user's own WhatsApp with a pre-
filled message
77                         (wa.me can't attach the local file, so the user adds the just-
downloaded reel). */}
78                     <a
79                         href={`https://wa.me/?text=${encodeURIComponent(t("result.shareText"))}`}
80                         target="_blank"
81                         rel="noopener noreferrer"
82                         data-testid="whatsapp-share"
83                         className="inline-flex min-h-11 items-center gap-1.5 rounded-full border
border-green/50 px-4 py-2 text-sm font-semibold text-green hover:bg-green/5"
84                     >
85                         <span aria-hidden="true">?</span>{t("result.share")}
86                     </a>
87                 </div>
88             </div>
89         ) : (
90             <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
91                 {build.error}
92             </p>
93         )}
94     </section>

```



```
95     );
96   }
97
```

16. user (2026-06-22T18:49:48.866Z)

```
1    // i18n scaffold (tracker B1) ? the shared localization foundation for KhelSutra Studio.
2    //
3    // Per docs/archives/FRIENDLY_AND_MULTILINGUAL.md + the engine's I18N_PLAN.md:
4    //   - keys, never prose (t("app.title"), enforced later by a no-bare-strings lint, B2);
5    //   - ICU MessageFormat for interpolation/plurals (Kannada/Hindi plural rules live in
the catalog,
6    //     not the code ? replaces the SPA's hand-rolled `count > 1 ? "s" : ""`);
7    //   - one negotiation order shared with the marketing site (B3);
8    //   - always-fallback to `en` so a missing translation degrades to English, never a raw
key.
9    import i18n from "i18next";
10   import { initReactI18next } from "react-i18next";
11   import LanguageDetector from "i18next-browser-languagedetector";
12   import ICU from "i18next-icu";
13   // Catalogs are HOISTED to repo-root /locales (D-CATALOG-HOME) ? the single source of
truth shared
14   // by this SPA and the marketing site's t() shim (tracker B3). Vite resolves the
workspace root via
15   // the repo `.git` marker, so importing the parent-dir /locales works in dev, build, and
viteest.
16   import enCommon from ".././../locales/en/common.json";
17   import knCommon from ".././../locales/kn/common.json";
18   import hiCommon from ".././../locales/hi/common.json";
19   import esCommon from ".././../locales/es/common.json";
20   import daCommon from ".././../locales/da/common.json";
21   import idCommon from ".././../locales/id/common.json";
22
23   // Supported locales: en/kn/hi foundation (FRIENDLY D6) + MULTILINGUAL_EXPANSION Phase A
(es/da/id,
24   // Latin, catalog-only). Anything else falls back to `en`.
25   export const SUPPORTED_LOCALES = ["en", "kn", "hi", "es", "da", "id"] as const;
26   export type Locale = (typeof SUPPORTED_LOCALES)[number];
27   export const DEFAULT_LOCALE: Locale = "en";
28
29   // ONE persisted-preference key, shared with the marketing site's t() shim (D9 / tracker
B3), so a
30   // visitor's language choice carries across both surfaces.
31   export const LANG_STORAGE_KEY = "khelsutra.lang";
32
33   // Catalogs live in repo-root /locales (D-CATALOG-HOME), consumed by both surfaces. `kn`
+ `hi`
34   // shipped (B5/B6, AI-drafted ? native review pending). A missing key in any locale
still falls back
35   // to `en` ? the by-design invariant.
36   const resources = {
37     en: { common: enCommon },
38     kn: { common: knCommon },
39     hi: { common: hiCommon },
40     es: { common: esCommon },
41     da: { common: daCommon },
42     id: { common: idCommon },
43   } as const;
44
45   void i18n
46     .use(ICU)
47     .use(LanguageDetector)
48     .use(initReactI18next)
49     .init({
50       resources,
```

```

51     fallbackLng: DEFAULT_LOCALE,
52     supportedLngs: [...SUPPORTED_LOCALES],
53     nonExplicitSupportedLngs: true, // en-IN -> en
54     defaultNS: "common",
55     ns: ["common"],
56     interpolation: { escapeValue: false }, // React already escapes
57     returnNull: false,
58     // Negotiation order (I18N_PLAN §2.3): persisted pref -> ?lang= -> browser -> en.
59     detection: {
60       order: ["localStorage", "querystring", "navigator"],
61       lookupLocalStorage: LANG_STORAGE_KEY,
62       lookupQuerystring: "lang",
63       caches: ["localStorage"],
64     },
65     react: { useSuspense: false },
66   });
67
68   // Keep <html lang> in sync with the active locale (screen readers + SEO use it).
Reflects the
69   // resolved content language, so an `en` fallback for an absent locale still reads as
`en`.
70   function syncDocumentLang(lng: string) {
71     if (typeof document !== "undefined") {
72       document.documentElement.lang = (i18n.resolvedLanguage || lng ||
DEFAULT_LOCALE).split("-")[0];
73     }
74   }
75   i18n.on("languageChanged", syncDocumentLang);
76   syncDocumentLang(i18n.language ?? DEFAULT_LOCALE);
77
78   export default i18n;
79

```

17. assistant (2026-06-22T18:49:52.239Z)

Now let me read the remaining UI components: SetupWizard, QueryBar, TapFilters, RallyGrid, SelectionFooter, LanguageSwitcher, and the types file.

18. user (2026-06-22T18:49:53.247Z)

```

1      // First-run setup wizard (PACKAGING_PLAN.md P2c). Three quick steps ? AI key / compute
/ videos ?
2      // rendered from the engine's GET /api/capabilities (an honest probe of what THIS box
can do).
3      // Localized (en/kn/hi/es/da/id). The Gemini-key step is GUIDED setup, not auto-save:
the engine has
4      // no key-write endpoint yet, so we tell the user how to add the key (offline `motion`
works with none).
5      import { useEffect, useState } from "react";
6      import { useTranslation } from "react-i18next";
7      import { getCapabilities } from "../api/client";
8      import type { EngineCapabilities } from "../api/types";
9      import { log, newCorrelationId } from "../lib/log";
10
11     /** localStorage flag ? distinct from the language key ? so the wizard shows ONCE per
browser. */
12     export const WIZARD_DONE_KEY = "kheIsutra.wizard.completed";
13
14     export function isWizardDone(): boolean {
15       try {
16         return localStorage.getItem(WIZARD_DONE_KEY) === "true";
17       } catch {
18         return false; // private-mode / no storage ? show it (harmless; user can skip)
19       }
20     }

```

```

21
22     const STEPS = 3;
23
24     function KeyStep({ caps }: { caps: EngineCapabilities | null }) {
25         const { t } = useTranslation();
26         const present = caps?.gemini_key_present === true;
27         return (
28             <div>
29                 <h3 className="font-semibold">{t("wizard.key.title")}</h3>
30                 {present ? (
31                     <p className="mt-2 font-medium text-green">{t("wizard.key.connected")}</p>
32                 ) : (
33                     <>
34                         <p className="mt-2 text-ink/80">{t("wizard.key.missing")}</p>
35                         <p className="mt-2 text-sm text-ink/70">{t("wizard.key.how")}</p>
36                         <a
37                             href="https://aistudio.google.com/apikey"
38                             target="_blank"
39                             rel="noreferrer"
40                             className="mt-2 inline-flex min-h-11 items-center text-sm font-medium text-
green underline"
41                             >
42                                 {t("wizard.key.getKey")}
43                             </a>
44                         </>
45                     </div>
46                 )};
47     };
48 }
49
50     function ComputeStep({ caps }: { caps: EngineCapabilities | null }) {
51         const { t } = useTranslation();
52         const ffmpegOk = caps?.ffmpeg?.ffmpeg === true && caps?.ffmpeg?.ffprobe === true;
53         const gpu = caps?.gpu?.nvidia_smi_present === true;
54         const detector = caps?.default_segmenter ?? "motion";
55         return (
56             <div>
57                 <h3 className="font-semibold">{t("wizard.compute.title")}</h3>
58                 <p className="mt-2 text-ink/80">{t("wizard.compute.local")}</p>
59                 <ul className="mt-3 space-y-1 text-sm text-ink/70">
60                     <li>{t("wizard.compute.detector", { name: detector })}</li>
61                     <li>{ffmpegOk ? t("wizard.compute.ffmpegOk") :
t("wizard.compute.ffmpegMissing")}</li>
62                     <li>{gpu ? t("wizard.compute.gpu") : t("wizard.compute.noGpu")}</li>
63                 </ul>
64             </div>
65         );
66     }
67
68     function VideosStep() {
69         const { t } = useTranslation();
70         return (
71             <div>
72                 <h3 className="font-semibold">{t("wizard.videos.title")}</h3>
73                 <p className="mt-2 text-ink/80">{t("wizard.videos.body")}</p>
74                 <p className="mt-2 font-medium text-green">{t("wizard.videos.ready")}</p>
75             </div>
76         );
77     }
78
79     export function SetupWizard({ onClose }: { onClose: () => void }) {
80         const { t } = useTranslation();
81         const [step, setStep] = useState(0);
82         const [caps, setCaps] = useState<EngineCapabilities | null>(null);
83         const [loading, setLoading] = useState(true);

```

```

84     const [offline, setOffline] = useState(false);
85
86     useEffect(() => {
87         let alive = true;
88         const cid = newCorrelationId();
89         getCapabilities(cid)
90             .then((c) => {
91                 if (!alive) return;
92                 setCaps(c);
93                 setLoading(false);
94             })
95             .catch((err) => {
96                 if (!alive) return;
97                 // Engine not up yet ? don't assert false capabilities; offer a graceful skip.
98                 setOffline(true);
99                 setLoading(false);
100                 log("warn", "wizard.capabilities_unavailable", { cid, error: String(err) });
101             });
102         return () => {
103             alive = false;
104         };
105     }, []);
106
107     const finish = () => {
108         try {
109             localStorage.setItem(WIZARD_DONE_KEY, "true");
110         } catch {
111             /* private mode: still close (the wizard just reappears next visit) */
112         }
113         onClose();
114     };
115
116     const stepping = !loading && !offline; // the 3-step flow only when capabilities
loaded
117     const isLast = step === STEPS - 1;
118
119     return (
120         <section
121             role="region"
122             aria-label={t("wizard.title")}
123             className="mt-8 rounded-2xl border border-ink/10 bg-white p-6 shadow-sm"
124         >
125             <div className="flex items-center justify-between gap-3">
126                 <h2 className="text-xl font-bold">{t("wizard.title")}</h2>
127                 <button
128                     type="button"
129                     onClick={finish}
130                     className="min-h-11 rounded-lg px-3 text-sm text-ink/60 hover:text-ink"
131                 >
132                     {t("wizard.skip")}
133                 </button>
134             </div>
135             <p className="mt-1 text-sm text-ink/70">{t("wizard.intro")}</p>
136             {stepping && (
137                 <p className="mt-1 text-xs font-medium text-ink/50">{t("wizard.step", { n: step
+ 1 })}</p>
138             )}
139
140             <div className="mt-5 min-h-[128px]" aria-live="polite">
141                 {loading && <p className="text-ink/60">{t("wizard.checking")}</p>}
142                 {!loading && offline && <p className="text-amber-700">{t("wizard.offline")}</p>}
143                 {stepping && step === 0 && <KeyStep caps={caps} />}
144                 {stepping && step === 1 && <ComputeStep caps={caps} />}
145                 {stepping && step === 2 && <VideosStep />}
146             </div>

```

```

147
148     <div className="mt-6 flex items-center justify-between">
149         {stepping ? (
150             <>
151                 <button
152                     type="button"
153                     disabled={step === 0}
154                     onClick={() => setStep((s) => Math.max(0, s - 1))}
155                     className="min-h-11 rounded-lg px-4 text-sm font-medium text-ink
disabled:opacity-40"
156                 >
157                     {t("wizard.back")}
158                 </button>
159                 <button
160                     type="button"
161                     onClick={() => (isLast ? finish() : setStep((s) => Math.min(STEPS - 1, s +
162                         1)))}
163                     className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
164                 >
165                     {isLast ? t("wizard.done") : t("wizard.next")}
166                 </button>
167             ) : offline ? (
168                 <button
169                     type="button"
170                     onClick={finish}
171                     className="min-h-11 rounded-lg bg-green px-5 text-sm font-semibold text-
white"
172                 >
173                     {t("wizard.done")}
174                 </button>
175             ) : null}
176         </div>
177     </section>
178 );
179 }
180

```

19. user (2026-06-22T18:49:53.656Z)

```

1 // Types mirror the engine's verified contract (badminton-highlight-indexer).
2 // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3 // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5 export interface RallyMetadata {
6     /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7     shot_count?: number;
8     /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9     shot_count_confidence?: string;
10    detector?: string;
11    [key: string]: unknown;
12 }
13
14 export interface RallySegment {
15     /** The integer the UI selects and passes to /api/compile as segment_ids. */
16     id: number;
17     start_time: number;
18     end_time: number;
19     duration: number;
20     /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21     server: string | null;
22     /** motion-segmenter FABRICATION (random) ? DO NOT surface. */

```

```

23     receiver: string | null;
24     metadata: RallyMetadata;
25     /** motion-only fabricated sub-events ? DO NOT surface. */
26     events?: unknown[];
27 }
28
29 export interface QueryResponse {
30     play_segments: RallySegment[];
31 }
32
33 export interface CompileResponse {
34     status: string;
35     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
36     filepath: string;
37 }
38
39 /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
40     * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |
41     * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
42     * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
43 export interface CompileResult {
44     status: string;
45     filepath?: string;
46     download_url?: string;
47     message?: string;
48     video_id?: string;
49 }
50
51 export interface EngineConfig {
52     stitching?: {
53         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
54         min_shot_count?: number;
55         [key: string]: unknown;
56     };
57     [key: string]: unknown;
58 }
59
60 /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
61     * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-booleans only (never the Gemini
key value).
62     * Every field optional/defensive: a missing field degrades the wizard gracefully. */
63 export interface EngineCapabilities {
64     api_version?: number;
65     engine_version?: string;
66     segmenters?: string[];
67     default_segmenter?: string;
68     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
69     gpu?: { nvidia_smi_present?: boolean };
70     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
71     gemini_key_present?: boolean;
72     wasb_weights_present?: boolean;
73     /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
74     * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
75     deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
76     [key: string]: unknown;

```

```

77     }
78
79     /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
80     export interface ProcessResponse {
81         job_id: string;
82         /** Always "queued" at submit. */
83         status: string;
84         /** Relative poll path, e.g. "/api/jobs/<id>". */
85         poll: string;
86     }
87
88     /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
89     * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
90     export interface UploadInitResponse {
91         upload_id: string;
92         /** Per-part ceiling in MB ? slice the file into parts this big. */
93         max_part_mb: number;
94         next_part: string;
95     }
96     export interface UploadPartResponse {
97         upload_id: string;
98         received_parts: number;
99         received_bytes: number;
100     }
101     export interface UploadCompleteResponse {
102         /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
103         path: string;
104         video_id: string;
105         size_bytes: number;
106     }
107
108     /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
109     * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
110     * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
shaped
111     * FAILURE ? the UI must not treat it as a load. */
112     export interface JobStatus {
113         job_id: string;
114         status: "queued" | "running" | "done" | "failed";
115         /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
116         progress?: string | null;
117         /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
118         video_id?: string | null;
119         /** Execution error (worker crash / unexpected) when status === "failed". */
120         error?: string | null;
121         result?: { status?: string; video_id?: string; message?: string } | null;
122     }
123

```

20. user (2026-06-22T18:49:54.054Z)

```

1     // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2     // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3     // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4     // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
5     import { useTranslation, Trans } from "react-i18next";

```

```

6      import { mmss } from "../lib/format";
7      import type { FloorRisk } from "../lib/floor";
8
9      interface Props {
10         count: number;
11         totalSec: number;
12         reelName: string;
13         onReelNameChange: (v: string) => void;
14         onBuild: () => void;
15         /** A compile is in flight (Build wired in P5). */
16         building?: boolean;
17         /** Non-null when some selected rallies sit below the engine floor. */
18         warning: FloorRisk | null;
19     }
20
21     export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
22         const { t } = useTranslation();
23         const buildDisabled = count === 0 || building || warning?.allDropped === true;
24
25         return (
26             <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
27                 {warning && (
28                     <div
29                         role="status"
30                         data-testid="floor-warning"
31                         className={
32                             "border-b px-6 py-2 text-sm " +
33                             (warning.allDropped
34                                 ? "border-red-200 bg-red-50 text-red-800"
35                                 : "border-amber-200 bg-amber-50 text-amber-900")
36                         }
37                     >
38                         {warning.allDropped ? (
39                             count === 1 ? (
40                                 <Trans i18nKey="footer.floorOne" values={{ min: warning.minShotCount }}
components={{ b: <strong /> }} />
41                             ) : (
42                                 <Trans i18nKey="footer.floorAll" values={{ count, min:
warning.minShotCount }} components={{ b: <strong /> }} />
43                             )
44                             ) : (
45                                 <>
46                                     <strong>{t("footer.floorSomeLead", { count: warning.atRisk.length
})}</strong>{" "}
47                                     <Trans i18nKey="footer.floorSomeRest" values={{ min: warning.minShotCount
}} components={{ code: <code /> }} />
48                                 </>
49                             )}
50                         </div>
51                     )}
52
53                     <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
54                         <span className="text-sm text-ink/80" data-testid="selection-summary">
55                             <strong className="text-ink">{t("footer.count", { count })}</strong>
56                             {t("footer.summaryRest", { total: mmss(totalSec) })}
57                         </span>
58
59                         <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
60                             <span className="hidden sm:inline">{t("footer.reelName")}</span>
61                             <input
62                                 type="text"
63                                 value={reelName}
64                                 onChange={(e) => onReelNameChange(e.target.value)}

```



```

64         placeholder={t("footer.reelNamePlaceholder")}
65         aria-label={t("footer.reelName")}
66         className="min-h-11 w-40 rounded-lg border border-black/15 bg-white px-3
py-1.5 text-ink outline-none focus:border-green"
67     />
68     </label>
69
70     <button
71         type="button"
72         onClick={onBuild}
73         disabled={buildDisabled}
74         className="min-h-11 rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
75     >
76         {building ? t("footer.building") : t("footer.build", { count })}
77     </button>
78 </div>
79 </footer>
80 );
81 }
82

```

21. assistant (2026-06-22T18:49:57.442Z)

Now let me read the QueryBar, TapFilters, RallyGrid, and LanguageSwitcher components to complete the picture of the rally selection/query screens.

22. user (2026-06-22T18:49:58.391Z)

```

1      // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2      // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3      // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4      // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5      // greyed out as "not available yet" ? never silently faked.
6      import { useMemo, useState } from "react";
7      import type { FormEvent } from "react";
8      import { useTranslation, Trans } from "react-i18next";
9      import { parseQuery, summarize } from "../lib/query";
10     import type { ParsedQuery } from "../lib/query";
11
12     interface Props {
13         /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
selection).
14         * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
the trace). */
15         onApply: (q: ParsedQuery, source: "typed" | "example") => void;
16     }
17
18     // Each example is a real, parseable query ? tapping one fills the box AND applies it.
Led by rally
19     // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
not featured
20     // here (a test pins that every example parses to a recognized, fully-available query).
21     export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
22
23     // Parsed-pill chips: legible on a phone (text-sm, not text-xs) ? A4.
24     const CHIP = "rounded-full px-3 py-1 text-sm font-medium";
25
26     export function QueryBar({ onApply }: Props) {
27         const { t, i18n } = useTranslation();
28         // The example chips are ENGLISH query syntax (the parser is English-only). For a non-

```

English user

```
29      // they're an untappable wall ? hide them and let the localized TapFilters (A9) own
the tap path.
30      const isEnglish = (i18n.resolvedLanguage ?? i18n.language ?? "en").split("-")[0] ===
"en";
31      const [value, setValue] = useState("");
32      const parsed = useMemo(() => parseQuery(value), [value]);
33      const clauses = summarize(parsed);
34      const typed = value.trim().length > 0;
35
36      const applyText = (text: string) => {
37          setValue(text);
38          onApply(parseQuery(text), "example");
39      };
40      const submit = (e: FormEvent) => {
41          e.preventDefault();
42          if (parsed.recognized) onApply(parsed, "typed");
43      };
44
45      return (
46          <div>
47              <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
48                  <input
49                      type="text"
50                      value={value}
51                      onChange={(e) => setValue(e.target.value)}
52                      placeholder={t("query.placeholder")}
53                      aria-label={t("query.ariaLabel")}
54                      className="min-h-11 min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green"
55                  />
56                  <button
57                      type="submit"
58                      disabled={!parsed.recognized}
59                      className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
60                  >
61                      {t("query.apply")}
62                  </button>
63              </form>
64
65              {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
*/}
66              <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
items-center gap-1.5">
67                  {clauses.map((c) => (
68                      <span key={c} className={CHIP + " bg-green/15 text-green">
69                          {c}
70                      </span>
71                  ))}
72                  {parsed.unavailable.map((u) => (
73                      <span
74                          key={u.token}
75                          title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
76                          className={CHIP + " bg-black/5 text-ink/45"}
77                      >
78                          <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
79                      </span>
80                  ))}
81                  {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
82                      <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
83                  )}
84                  {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
85              </div>
86          </div>
```

```

87      {/* Surface the "not available yet" reason as VISIBLE text (the per-chip title is
hover-only,
88      so it never appears on a phone) ? A4. */}
89      {parsed.unavailable.length > 0 && (
90        <p className="mt-1.5 text-xs text-ink/50">{t("query.notAvailableHelp")}</p>
91      )}
92
93      {isEnglish && (
94        <div className="mt-3 flex flex-wrap items-center gap-2 text-sm">
95          <span className="text-ink/40">{t("query.try")}</span>
96          {EXAMPLES.map((ex) => (
97            <button
98              key={ex}
99              type="button"
100              onClick={() => applyText(ex)}
101              className="inline-flex min-h-11 items-center rounded-full border border-
black/15 px-4 text-sm hover:border-green"
102            >
103              {ex}
104            </button>
105          )}}
106        </div>
107      )}
108
109      <p className="mt-2 text-xs text-ink/40">
110        <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
111      </p>
112    </div>
113  );
114 }
115

```

23. user (2026-06-22T18:49:59.343Z)

```

1      // Localized tap-only filter controls (tracker A9). The query bar PARSES ENGLISH text (a
documented
2      // limit) ? so a Kannada/Hindi/? user who can't type English needs to filter by TAPPING.
Each control
3      // builds the SAME ParsedQuery the parser would for its English equivalent ? directly,
no English
4      // round-trip ? and applies it through the shared onApply path. A test pins tap == type
so a control
5      // can never silently drift from the parser. Length-led + fully-available only (never
shot-count).
6      import { useTranslation } from "react-i18next";
7      import type { ParsedQuery } from "../lib/query";
8
9      interface TapFilter {
10        /** i18n label key (localized) ? the user sees this, not the English query. */
11        key: string;
12        /** The equivalent typed query; the tap MUST select identically to typing this (pinned
by a test). */
13        english: string;
14        parts: Pick<ParsedQuery, "filters" | "sort" | "limit">;
15      }
16
17      export const TAP_FILTERS: TapFilter[] = [
18        { key: "tap.longest", english: "longest first", parts: { filters: [], sort: { key:
"duration", dir: "desc" } } },
19        { key: "tap.shortest", english: "shortest first", parts: { filters: [], sort: { key:
"duration", dir: "asc" } } },
20        { key: "tap.over10", english: "over 10s", parts: { filters: [{ field: "duration", op:
">", value: 10 } ] } },
21        { key: "tap.over20", english: "over 20s", parts: { filters: [{ field: "duration", op:

```

```

">", value: 20 }} ] },
22     { key: "tap.top5", english: "top 5", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 5 } },
23     { key: "tap.top10", english: "top 10", parts: { filters: [], sort: { key: "duration",
dir: "desc" }, limit: 10 } },
24 ];
25
26 interface Props {
27     /** Same handler the QueryBar uses ? App runs runQuery + writes the shared selection.
*/
28     onApply: (q: ParsedQuery, source: "typed" | "example") => void;
29 }
30
31 export function TapFilters({ onApply }: Props) {
32     const { t } = useTranslation();
33     // ?44px tap targets (A4) so the no-type path is comfortable on a phone.
34     const BTN = "inline-flex min-h-11 items-center rounded-full border border-black/15
px-4 text-sm hover:border-green";
35
36     return (
37         <div className="mt-3" data-testid="tap-filters">
38             <span className="text-xs text-ink/40">{t("tap.title")}</span>
39             <div className="mt-1.5 flex flex-wrap gap-2">
40                 {TAP_FILTERS.map((f) => (
41                     <button
42                         key={f.key}
43                         type="button"
44                         className={BTN}
45                         onClick={() =>
46                             // raw mirrors the equivalent typed query so tap/type traces are directly
comparable
47                             // (source="example" still flags it as a chip, not free-text).
48                             onApply(
49                                 { raw: f.english, filters: f.parts.filters, sort: f.parts.sort, limit:
f.parts.limit, unavailable: [], recognized: true },
50                                 "example",
51                             )
52                         }
53                     >
54                         {t(f.key)}
55                     </button>
56                 ))}
57             </div>
58         </div>
59     );
60 }
61

```

24. user (2026-06-22T18:49:59.813Z)

```

1     // The selectable rally grid ? one card per play_segment, HONEST fields only:
2     // ordinal (by start order), mm:ss start, and rally LENGTH (the trusted axis). A shot-
count chip
3     // is shown ONLY when present, and clearly flagged EXPERIMENTAL ? shot detection is
still under
4     // development and not yet reliable, so length is what we lead with (owner 2026-06-20).
5     // Never renders server/receiver/events (motion-only fabrications,
PER_RALLY_SELECTION.md §4).
6     import { useTranslation } from "react-i18next";
7     import type { RallySegment } from "../api/types";
8     import { mmss } from "../lib/format";
9
10    interface Props {
11        rallies: RallySegment[];
12        isSelected: (id: number) => boolean;

```

```

13     onToggle: (id: number) => void;
14     /** Stable chronological label for a rally (by start_time), so re-sorting via a query
15      * doesn't renumber the cards. Falls back to display position when absent. */
16     ordinalOf?: (id: number) => number;
17   }
18
19   export function RallyGrid({ rallies, isSelected, onToggle, ordinalOf }: Props) {
20     const { t } = useTranslation();
21     if (rallies.length === 0) {
22       return <p className="text-ink/60 text-sm">{t("grid.empty")}</p>;
23     }
24     return (
25       <ul className="grid grid-cols-[repeat(auto-fill,minmax(150px,1fr))] gap-3 list-none
p-0 m-0">
26         {rallies.map((r, i) => {
27           const selected = isSelected(r.id);
28           const n = ordinalOf ? ordinalOf(r.id) : i + 1;
29           const shot = r.metadata?.shot_count;
30           const conf = r.metadata?.shot_count_confidence;
31           return (
32             <li key={r.id}>
33               <button
34                 type="button"
35                 onClick={() => onToggle(r.id)}
36                 aria-pressed={selected}
37                 aria-label={t(selected ? "grid.inReel" : "grid.notInReel", { n })}
38                 className={
39                   "w-full text-left rounded-xl border p-3 transition " +
40                   (selected
41                     ? "border-green bg-green/10"
42                     : "border-black/10 bg-white hover:border-green/60")
43                 }
44               >
45                 <div className="flex items-center justify-between">
46                   <span className="font-semibold text-ink">{t("grid.rally", { n })}</span>
47                   <span
48                     aria-hidden="true"
49                     className={
50                       "inline-flex h-5 w-5 items-center justify-center rounded-full text-
xs " +
51                       (selected ? "bg-green text-[#04140D]" : "border border-black/20
text-transparent")
52                     }
53                   >
54                     ?
55                   </span>
56                 </div>
57                 <div className="mt-1 text-sm text-ink/70">
58                   {mmss(r.start_time)} <span className="text-
ink/40">{t("grid.start")}</span> · <strong className="text-ink/90">{mmss(r.duration)}</strong>
<span className="text-ink/40">{t("grid.long")}</span>
59                 </div>
60                 {typeof shot === "number" && (
61                   <div
62                     title={t("grid.shotsTitle", {
63                       conf: conf && conf !== "Unknown" ? t("grid.shotsConfidence", { conf
64                     }) : "",
65                     })}
66                     className="mt-2 inline-block rounded-full bg-amber-50 px-2 py-0.5
text-xs text-amber-700"
67                   >
68                     {/* Confidence is shown VISIBLY (not only in the hover title) so a
phone user ?
69                     who never sees title= ? still gets the per-card confidence (A4).
*/}

```

```

69         {t("grid.shots", { count: shot })}
70         {conf && conf !== "Unknown" ? t("grid.shotsConfidence", { conf }) :
""}
71         </div>
72     )}
73 </button>
74 </li>
75 );
76 }}}
77 </ul>
78 );
79 }
80

```

25. user (2026-06-22T18:50:00.109Z)

```

1     import { useTranslation } from "react-i18next";
2     import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";
3
4     // Each locale's label is its own autonym, shown in its own script. Kannada renders
crisply once the
5     // bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system
Kannada font.
6     const LANG_LABEL_KEY: Record<Locale, string> = {
7         en: "lang.en",
8         kn: "lang.kn",
9         hi: "lang.hi",
10        es: "lang.es",
11        da: "lang.da",
12        id: "lang.id",
13    };
14
15    /**
16     * A persistent language switcher. Selecting a locale calls i18next's changeLanguage,
which the
17     * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-
priority signal on
18     * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state
via
19     * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
20     */
21    export function LanguageSwitcher() {
22        const { t, i18n } = useTranslation();
23        // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content
one:
24        // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada
(content
25        // still English until the kn catalog lands, B5) must still show Kannada as selected.
Compare on
26        // the base subtag so a detected "en-IN" highlights "en".
27        const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];
28
29        return (
30            <nav aria-label={t("lang.label")} className="flex items-center gap-1">
31                {SUPPORTED_LOCALES.map((loc) => {
32                    const isActive = active === loc;
33                    return (
34                        <button
35                            key={loc}
36                            type="button"
37                            lang={loc}
38                            aria-pressed={isActive}
39                            onClick={() => void i18n.changeLanguage(loc)}
40                            className={[
41                                "inline-flex min-h-11 items-center rounded-full px-4 text-sm font-semibold

```

```

transition",
42             isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
43             ].join(" "))}
44         >
45             {t(LANG_LABEL_KEY[loc])}
46         </button>
47     );
48     }}
49 </nav>
50 );
51 }
52

```

26. assistant (2026-06-22T18:50:19.012Z)

I now have a complete picture of the entire SPA. Let me verify the i18n catalog actually has the 6 locales present and check a couple of key facts (locale list, query.support copy mentioning shot-count caveat). Let me also confirm the locales directory exists with all 6.

27. user (2026-06-22T18:50:22.509Z)

```

da
en
es
hi
id
kn
---en keys sample---
ingest.compute.cloud: "In the cloud"
ingest.compute.local: "On my computer"
ingest.compute.cloudCost: "Cloud processing runs on a rented GPU ? faster for long videos, but
it may cost a little (often just a few cents per match)."
result.ready: "Your reel is ready"
result.download: "Download {filename}"

```

28. assistant (2026-06-22T18:50:26.900Z)

All 6 locales present and the compute-picker labels match exactly what the task described ("On my computer" / "In the cloud"). Let me verify the build-error test and the async compile (#108/#329) wiring are actually exercised, and confirm the `useUpload` test exists, to ground the "shipped" claim. Let me also confirm there's no real video player/preview gap.

29. user (2026-06-22T18:50:29.334Z)

```

ed33a51 feat(build): poll the async /api/compile job for the reel (#329) (#108)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages)
(#88)
0fae364 feat(web+site): A2 ? localized WhatsApp share for the reel (SPA + site) (#82)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process
(analytics) (#73)
ca619fb feat(web): T4 ? in-UI file upload (the runnable-tool keystone) (#66)
8bbc557 feat(web): VITE_API_BASE ? let the SPA target a split api.* origin (default same-origin
/api) (#64)
92a6c45 feat(web): A8 ? background-processing reassurance + icon+text status chips (#63)
---grep pollCompileJob refs in tests---
web/src/api/client.test.ts
web/src/components/build-error.test.tsx
---main.tsx---
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";
import ".i18n"; // initialize the i18next singleton before the app renders (tracker B1)
// Self-hosted OFL Noto fonts so Kannada/Hindi render crisply offline (tracker B4 / decision D7)

```

```
?
// no Google Fonts runtime dependency. unicode-range-scoped, so only Indic codepoints use them.
import "@fontsource-variable/noto-sans-kannada/index.css";
import "@fontsource-variable/noto-sans-devanagari/index.css";
import "../index.css";

const root = document.getElementById("root");
if (!root) throw new Error("#root not found");

createRoot(root).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

30. assistant (2026-06-22T18:51:29.700Z)

I have all the verified facts I need. The async compile poll (#108/#329), compute-picker (#104), wizard (#88), file upload (#66), and locale-to-watermark (#73) are all merged into master and covered by tests. Let me do one final check to confirm the WhatsApp share and download anchor are not gated behind anything, and verify the test suite for upload exists (already saw `useUpload.test.ts`, `upload.test.tsx`). I have enough to report.

Here is the verified, step-by-step user journey.

KhelSutra Studio SPA ? Verified User Journey

All claims below are grounded in `C:/Users/avidu/Projects/khelsutra/web/src` on `master` @ `bc4e7ba` (repo pulled clean, ff-only). The full flow is SHIPPED end-to-end (file upload ? process ? curate ? async build ? preview/download), with one honest caveat about reel preview (below).

Screen layout / orchestrator

`web/src/App.tsx` is the single-page orchestrator. Header (`App.tsx:222-229`) shows the "Khel?Sutra Studio" wordmark + the `LanguageSwitcher`. The body conditionally renders one of two halves based on whether a `videoId` is set (`App.tsx:55-57`, seeded from `?video=` URL param, also settable in-session):

- **No `videoId` ? front-half (ingest):** SetupWizard (first run only) ? IngestPanel.
- **`videoId` set ? back-half (curate):** rallies load, QueryBar/TapFilters, RallyGrid, SelectionFooter, CompileResult.

The two halves are bridged without a reload: `IngestPanel.onLoaded(id)` calls `setVideoId(id)` and rewrites `?video=<id>` via `history.replaceState` (`App.tsx:246-253`).

Step 1 ? First-run setup wizard (once per browser)

`web/src/components/SetupWizard.tsx`. Renders only when `!videoId && !wizardDone` (`App.tsx:242`); a `localStorage` flag `khelsutra.wizard.completed` shows it once (`SetupWizard.tsx:12-20`). It probes `GET /api/capabilities` (`client.ts:184`) and walks 3 honest steps (`SetupWizard.tsx:143-145`):

1. **KeyStep** ? is a Gemini key present? If not, links to `aistudio.google.com/apikey` (GUIDED setup only ? there is NO key-write endpoint; the header comment at `SetupWizard.tsx:1-4` states this explicitly; offline `motion` segmenter works with no key).
2. **ComputeStep** ? shows default segmenter, ffmpeg/ffprobe presence, GPU presence (`SetupWizard.tsx:50-66`).
3. **VideosStep** ? static "ready" copy (`SetupWizard.tsx:68-77`).

Back/Next/Done + Skip buttons (`SetupWizard.tsx:148-176`). If `api/capabilities` fails, it shows an offline notice + a single "Done" to dismiss (no false capability claims) (`SetupWizard.tsx:95-101, 142, 167-175`).

Step 2 ? Load a video (IngestPanel)

`web/src/components/IngestPanel.tsx`. Two convergent paths:

- **PRIMARY ? in-UI file upload (T4):** a native `<input type="file" accept=".mp4,.mov,...">` styled as a button (`IngestPanel.tsx:140-161`). On pick, `useUpload.start(file)` (`web/src/hooks/useUpload.ts`) chunks the file via `uploadFile()` ? `init` ? sequential `PUT part` ? `complete` (`client.ts:262-291`), honoring the engine's `max_part_mb` (default 95 MB, `client.ts:270`) to stay under ~100 MB edge limits. Shows a **determinate % progress bar** (`IngestPanel.tsx:190-209`), a Cancel button (AbortController-backed), and rejects 0-byte files before hitting the network (`useUpload.ts:38-41`). On completion the returned server **path** is handed to `submit(path, locale, sendCompute)` (`IngestPanel.tsx:52-54`).
- **SECONDARY ? paste a URI:** a text field accepting a `gs://`gcs:///`s3:///`` bucket URI or a host path (`IngestPanel.tsx:163-185`), submitted to the same back-half.

Both converge on `useIngestJob.submit` (`web/src/hooks/useIngestJob.ts`): `POST /api/process {video_path, locale, compute}` ? `202 {job_id}` (`client.ts:85-99`), then **recursive-setTimeout** poll of `GET /api/jobs/{id}` with 1.5x backoff, 1s?5s, 10-min ceiling (`useIngestJob.ts:15-18, 75-114`). Progress is shown as the engine's **coarse free-text phase words**, not a fake % (`useIngestJob.ts:106`, `IngestPanel.tsx:220-242`). A success-shaped failure (job `done` but `result.status=="failed"` or no `video_id`) is surfaced as `emptyResult`, never silently loaded (`useIngestJob.ts:88-100`). On success `onLoaded(videoId)` fires (`useIngestJob.ts:96-98`).

Compute picker (`IngestPanel.tsx:91-138`): shown ONLY if `capabilities.deployment.cloud_available === true` (`IngestPanel.tsx:37-45`). Two toggle buttons labeled "On my computer" / "In the cloud" (verified i18n: `ingest.compute.local`/.cloud``). Default = `local` (`IngestPanel.tsx:35`). Picking **cloud** reveals a cost note ("may cost a little, often just a few cents per match") + a **required acknowledgement checkbox**; until checked, `cloudBlocked` disables both submit surfaces and blocks dispatch ? no paid job runs without explicit consent (`IngestPanel.tsx:50, 57, 61, 67, 117-136`). The choice is sent as `compute` only when the picker is shown, else omitted ? server default (`IngestPanel.tsx:46-47`).

Step 3 ? Rallies load (curate screen appears)

When `videoId` is set, `App.tsx:72-102` calls `listRallies` ? `POST /api/query {video_id}` ? `play_segments` (`client.ts:107-114`). A `ModeBanner` (`App.tsx:289-316`) shows loading / loaded-count / friendly localized error / demo states. On load all rallies are **selected by default** (`App.tsx:85`, `useRallySelection`). Without `?video=`, the app shows 6 hardcoded **DEMO_RALLIES** (`App.tsx:35-42`) and a `mode.demo` banner.

Step 4 ? Find/query rallies

- **QueryBar** (`web/src/components/QueryBar.tsx`): an OFFLINE deterministic parser turns text like "top 15 longest"/"over 10s" into a selection (`QueryBar.tsx:32, 40-43`). A live "parsed-pill" shows how the text was understood; unsupported concepts (shot-type/player/score) render greyed-out as "not available yet" ? never faked (`QueryBar.tsx:66-91`). English example chips ("over 10s", "top 15 longest", ?) show **only for English users** (`QueryBar.tsx:21, 30, 93-107`).
- **TapFilters** (`web/src/components/TapFilters.tsx`): localized tap-only buttons (longest/shortest/over 10s/over 20s/top 5/top 10) for users who can't type English ? each builds the same `ParsedQuery` (`TapFilters.tsx:17-24, 40-57`).
- **Bulk controls** (`App.tsx:263-267`): select all / clear / invert.

Step 5 ? Pick rallies (RallyGrid)

`web/src/components/RallyGrid.tsx`: one selectable card per rally showing **honest fields only** ? stable chronological ordinal, mm:ss start, rally **length** (the trusted axis). A shot-count chip appears only when present and is flagged experimental; `server`/`receiver`/`events`` (motion fabrications) are never rendered (`RallyGrid.tsx:1-5, 57-72`). Tap toggles selection (`App.tsx:208-211`).

Step 6 ? Build the reel (async, #108/#329)

SelectionFooter (`web/src/components/SelectionFooter.tsx`, sticky) shows selection count/total, a reel-name input, and the Build button. Build is disabled when count=0, building, or all selected rallies are below the engine's `min_shot_count` floor (`SelectionFooter.tsx:23`). An **honest floor warning** surfaces rallies the engine would

silently drop, before build (`SelectionFooter.tsx:27-51`, fed by `floorRisk` from `GET /api/config` `min\_shot\_count`, `App.tsx:106-116`).

`App.onBuild` (`App.tsx:126-162`) ? `compileReel(videoId, ids, filename, cid, locale)`. This is now **ASYNC** (`client.ts:127-171`): `POST /api/compile` returns `202 + job\_id`, then `pollCompileJob` polls `GET /api/jobs/{id}` (1s?5s backoff, 10-min ceiling) until the ffmpeg stitch finishes ? deliberately to avoid the ~100s edge 524 a long stitch would otherwise hit (`client.ts:116-125, 146-171`). The **active UI locale is sent** so the engine localizes the reel's watermark (`App.tsx:150`, LW1). A failed/timed-out/no-URL compile throws and is mapped to friendly localized text (`client.ts:153-167`, `App.tsx:154-161`).

Step 7 ? Preview + download

CompileResult (`web/src/components/CompileResult.tsx`): a "building?" card with spinner+reassurance during the stitch (`CompileResult.tsx:26-33`). On success:

- An inline **<video controls playsInline>** previews the reel via `highlightUrl(videoId, filename)` ? `GET /api/highlights/{video\_id}/{filename}` (`CompileResult.tsx:61-67`, `client.ts:174-176`).
- A **Download** anchor (`<a download>`) (`CompileResult.tsx:69-75`).
- A **WhatsApp share** button (`wa.me` with localized pre-filled text; it can't attach the local file, so the user attaches the just-downloaded reel ? comment at `CompileResult.tsx:76-86`).

On failure it shows the engine's reason verbatim, never swallowed (`CompileResult.tsx:89-93`).

Localization ? 6 locales VERIFIED

`web/src/i18n/index.ts`: `SUPPORTED\_LOCALES = ["en", "kn", "hi", "es", "da", "id"]` (`index.ts:25`). All 6 catalogs exist at repo-root `locales/{en,kn,hi,es,da,id}/common.json` (verified on disk) and are imported (`index.ts:16-43`). Negotiation order: persisted pref ? `?lang=` ? browser ? `en` fallback (`index.ts:58-64`); `LanguageSwitcher (`web/src/components/LanguageSwitcher.tsx`) renders all 6 as autonym buttons in the header. Kannada/Hindi ship self-hosted Noto fonts (`web/src/main.tsx:6-8`).

Caveats / what is NOT as one might assume (verified)

- **Reel preview depends on browser codec support.** The `` src is the raw `/api/highlights/...` stream; nothing transcodes for browser playback. Download always works regardless.
- **Gemini key entry is guided, not in-app.** The wizard links out to get a key; there is no key-write endpoint ? the engine reads it from its own env (stated at `SetupWizard.tsx:1-4`).
- **The query parser is English-only.** Non-English users get TapFilters instead of the QueryBar's example chips (`QueryBar.tsx:30, 93`); this is by design, not a gap.
- **Demo mode is data-only.** Without `?video=`, six hardcoded demo rallies render and Build will fail with a localized "no video" message (`App.tsx:131-137`) ? demo rallies have no real `video\_id` to compile.
- **server/receiver/events** are never surfaced anywhere in the UI (motion-segmenter fabrications), per the type annotations (`api/types.ts:20-26`) and RallyGrid honesty rule.

Relevant files (all absolute):

- `C:/Users/avidu/Projects/khelsutra/web/src/App.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/IngestPanel.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/hooks/useUpload.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/hooks/useIngestJob.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/api/client.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/api/types.ts`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/CompileResult.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/SelectionFooter.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/SetupWizard.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/QueryBar.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/TapFilters.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/RallyGrid.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/components/LanguageSwitcher.tsx`
- `C:/Users/avidu/Projects/khelsutra/web/src/i18n/index.ts`
- `C:/Users/avidu/Projects/khelsutra/locales/{en,kn,hi,es,da,id}/common.json`